

Service-oriented Software Engineering (SoSE)

MAVEN intro

Prof. Marco Autili

Department of Information Engineering Computer Science and Mathematics

University of L'Aquila

marco.autili@univaq.it

<http://people.disim.univaq.it/marco.autili/>

What have we done?

How far have we gone?

- Implementation of our first WS without any supporting framework, only `javax.jws.*`, `javax.xml.ws.handler.*`, etc.
- Manual implementation of the client without considering the WSDL ☹
- Implementation of our first WS by using Apache CXF
 - Manual resolution of the library dependencies ☹
 - Automatic generation of Java classes from the WSDL by using the Apache CXF `wsdl2java` tool ☺
- Deployment and execution on Tomcat directly from within the Eclipse IDE

How far we will go with these slides

- Exploiting MAVEN to support the whole lifecycle of our project, from project structure to dependencies management, from source code to packaging, from deployment and execution to profiling for distribution, and much more
- More complex interaction via asynchronous communication

What is Maven?

Official Web site -- <http://maven.apache.org>

For the great majority of users, Maven is

- a xml-based build tool for Java-based project
- a tool used to build deployable artifacts from source code,
i.e., a tool offering preprocessing, compilation, packaging, testing, and distribution

Build engineers and project managers might refer to Maven as

- a project management tool
- a tool providing a superset of features with respect to older build tools such as **Make** and **Ant**
- a tool that, beyond build capabilities, runs reports, generates web sites, and facilitates communication among members of a working team

Suggestion -- visit <https://maven.apache.org/what-is-maven.html> (a summary follows)

What is Maven?

According to the Apache Maven official site Maven is a **project management tool** which encompasses

- a project object model
- a set of standards
- a project lifecycle
- a dependency management system
- logic for executing plugin goals at defined phases in a lifecycle

Maven is **not limited to Java**, but it is primarily designed for Java projects; support for **other languages is possible through plugins**, though often not optimal

Maven Standard Directory Layout

Having a common directory layout would allow for users familiar with one Maven project to immediately feel at home in another Maven project. The advantages are analogous to adopting a site-wide look-and-feel.

The next section documents the directory layout expected by Maven and the directory layout created by Maven. Please try to conform to this structure as much as possible; however, if you can't these settings can be overridden via the project descriptor.

src/main/java	Application/Library sources
src/main/resources	Application/Library resources
src/main/filters	Resource filter files
src/main/webapp	Web application sources
src/test/java	Test sources
src/test/resources	Test resources
src/test/filters	Test resource filter files
src/it	Integration Tests (primarily for plugins)
src/assembly	Assembly descriptors
src/site	Site
LICENSE.txt	Project's license
NOTICE.txt	Notices and attributions required by libraries that the project depends on
README.txt	Project's readme

Maven incorporates the concept of **Convention Over Configuration** by providing (built-in) sensible default behavior for projects. Thus, without any customization

- this project structure is assumed
- Maven also assumes, e.g., that you want the compile the byte-code to `${basedir}/target/classes` and then create a distributable JAR file in `${basedir}/target`
- Maven has a defined life-cycle and a set of common plugins that know how to build and assemble software

At the top level, files descriptive of the project: a `pom.xml` file. In addition, there are textual documents meant for the user to be able to read immediately on receiving the source: `README.txt`, `LICENSE.txt`, etc.

There are just two subdirectories of this structure: `src` and `target`. The only other directories that would be expected here are metadata like `CVS`, `.git` or `.svn`, and any subprojects in a multiproject build (each of which would be laid out as above).

The `target` directory is used to house all output of the build.

The `src` directory contains all of the source material for building the project, its site and so on. It contains a subdirectory for each type: `main` for the main build artifact, `test` for the unit test code and resources, `site` and so on.

Within artifact producing source directories (ie. `main` and `test`), there is one directory for the language `java` (under which the normal package hierarchy exists), and one for `resources` (the structure which is copied to the target classpath given the default resource definition).

If there are other contributing sources to the artifact build, they would be under other subdirectories: for example `src/main/antlr` would contain Antlr grammar definition files.

<http://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>

Maven's objectives (1/3)

There are several areas of concern that Maven deals with

- Making the build process easy
- Providing uniform build system
- Providing quality project information
- Providing guidelines for best practices development
- Allowing transparent migration to new features

<http://maven.apache.org/what-is-maven.html>

Maven's objectives (2/3)

Making the build process easy

- while using Maven doesn't eliminate the need to know about the underlying mechanisms, Maven does provide a lot of shielding from the details

Providing a uniform build system

- Maven allows a project to build using its Project Object Model (POM) and a set of plugins that are shared by all projects using Maven, providing a uniform build system
- once you familiarize yourself with how one Maven project builds, you automatically know how all Maven projects build, saving you immense amounts of time when trying to navigate many projects

<http://maven.apache.org/what-is-maven.html>

Maven's objectives (3/3)

Providing quality project information

- Maven provides plenty of useful **project information** that is in part **taken from your POM** and in part **generated from your project's sources**
- for example, Maven can provide
 - **change log document** created directly from source control
 - **cross referenced sources**
 - **mailing lists**
- Maven can also provide
 - **dependency list**
 - **unit test reports including coverage**

See also <http://maven.apache.org/what-is-maven.html>

- **providing guidelines for best practices development**
- **allowing transparent migration to new features**

<http://maven.apache.org/what-is-maven.html>

Project Object Model (POM)

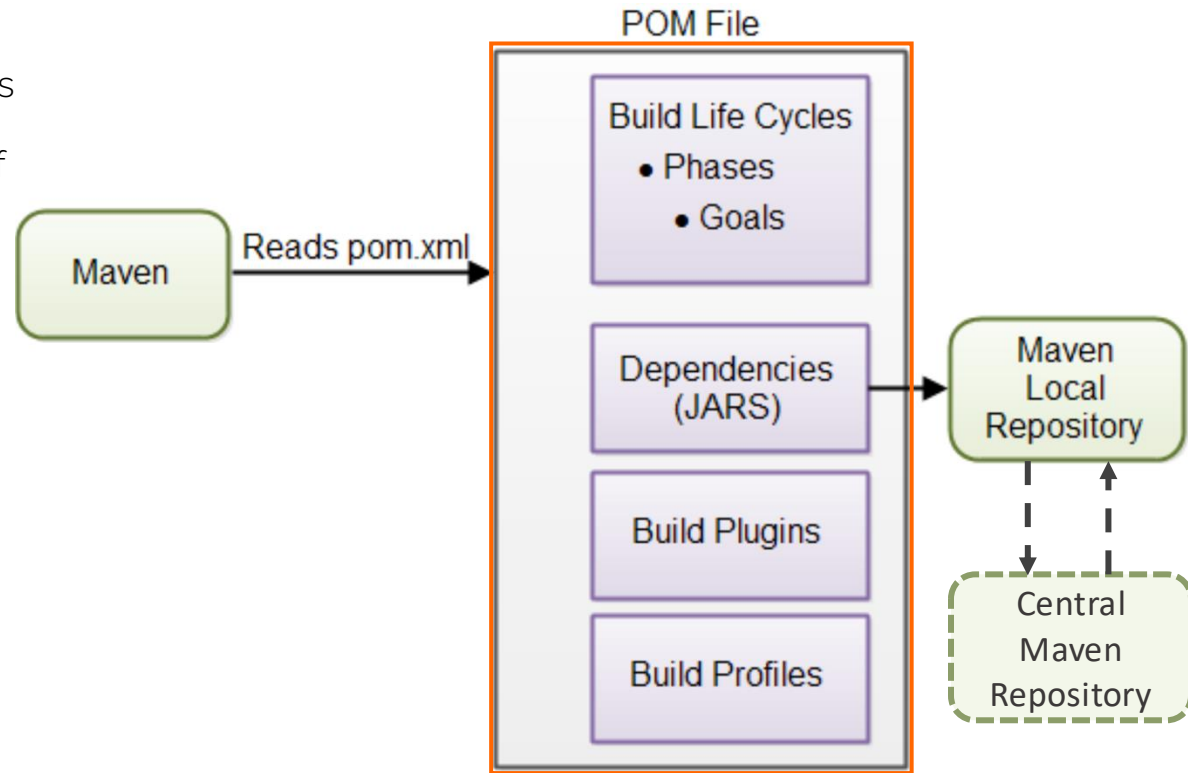
It is an XML file (`pom.xml`)

- It can be said it is a **Conceptual Model** for projects
- It provides a full description of a software project

It describes the resources of the project

- source code, project information, test source, etc.
- external dependencies (JAR files)

It should be located in the root directory of your project



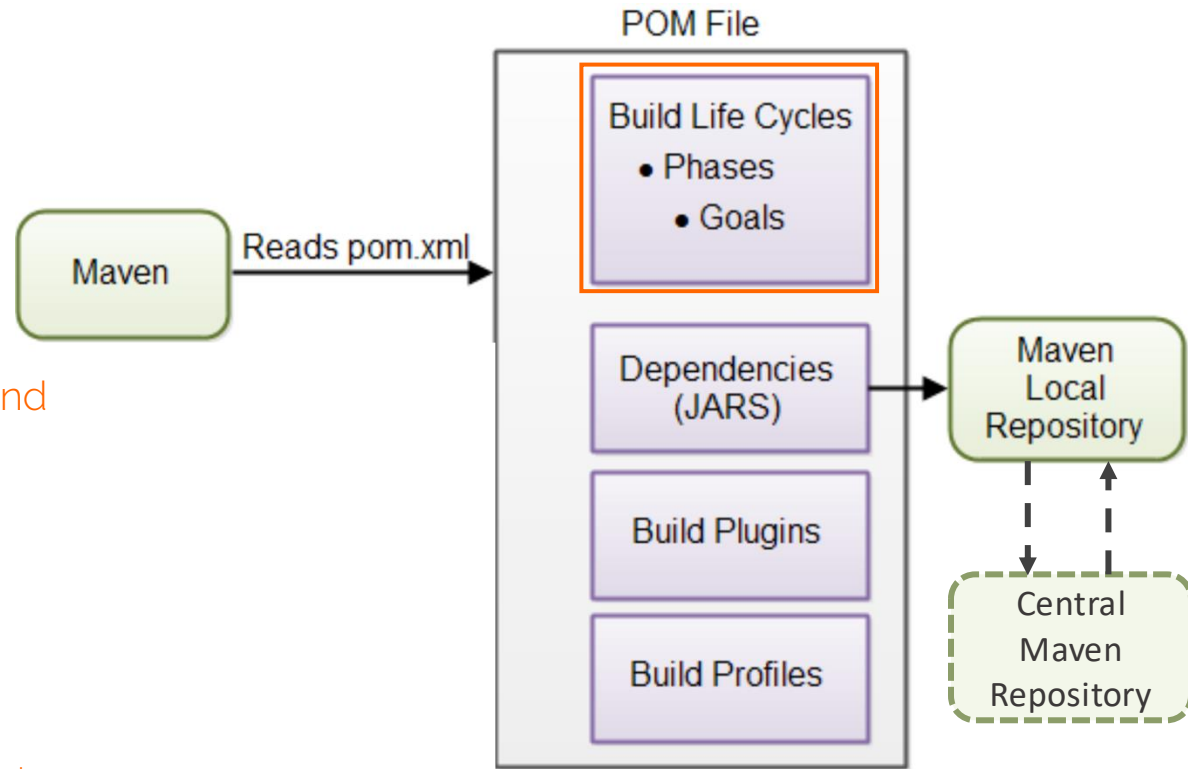
Build Lifecycles, Phases and Goals

Build process is split up into Build Life Cycles, Phases and Goals

- a build life cycle consists of a sequence of build phases
- each build phase consists of a sequence of goals

Maven is run by passing it a command (name of a build lifecycle, phase or goal)

- if a build lifecycle is executed, all build phases in that life cycle are executed
- if a build phase is executed, all build phases before it in the pre-defined sequence of build phases are executed too



Dependencies

One of the first goals Maven executes is to **check the dependencies** needed by your project

- dependencies are external **JAR files** that your project uses

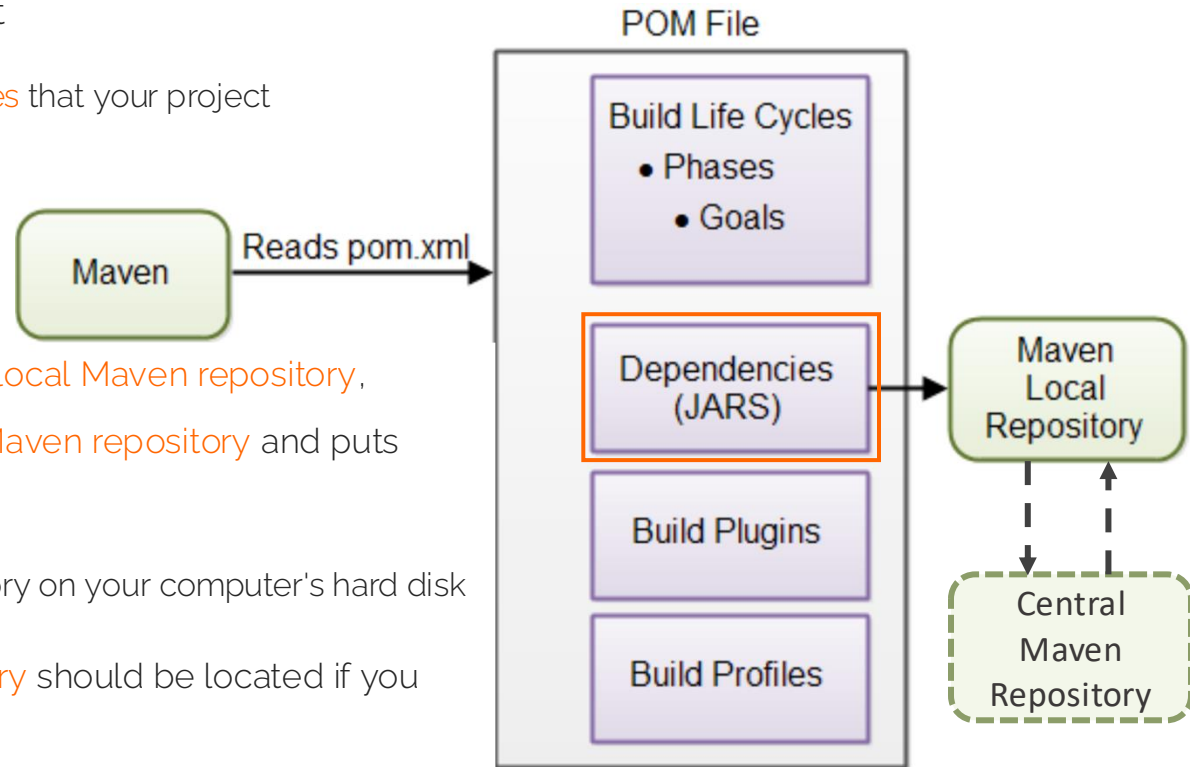
If the dependencies are **not found in the local Maven repository**, Maven downloads them from a **central Maven repository** and puts them in your local repository

- the local repository is just a directory on your computer's hard disk

You can **specify where the local repository** should be located if you want to

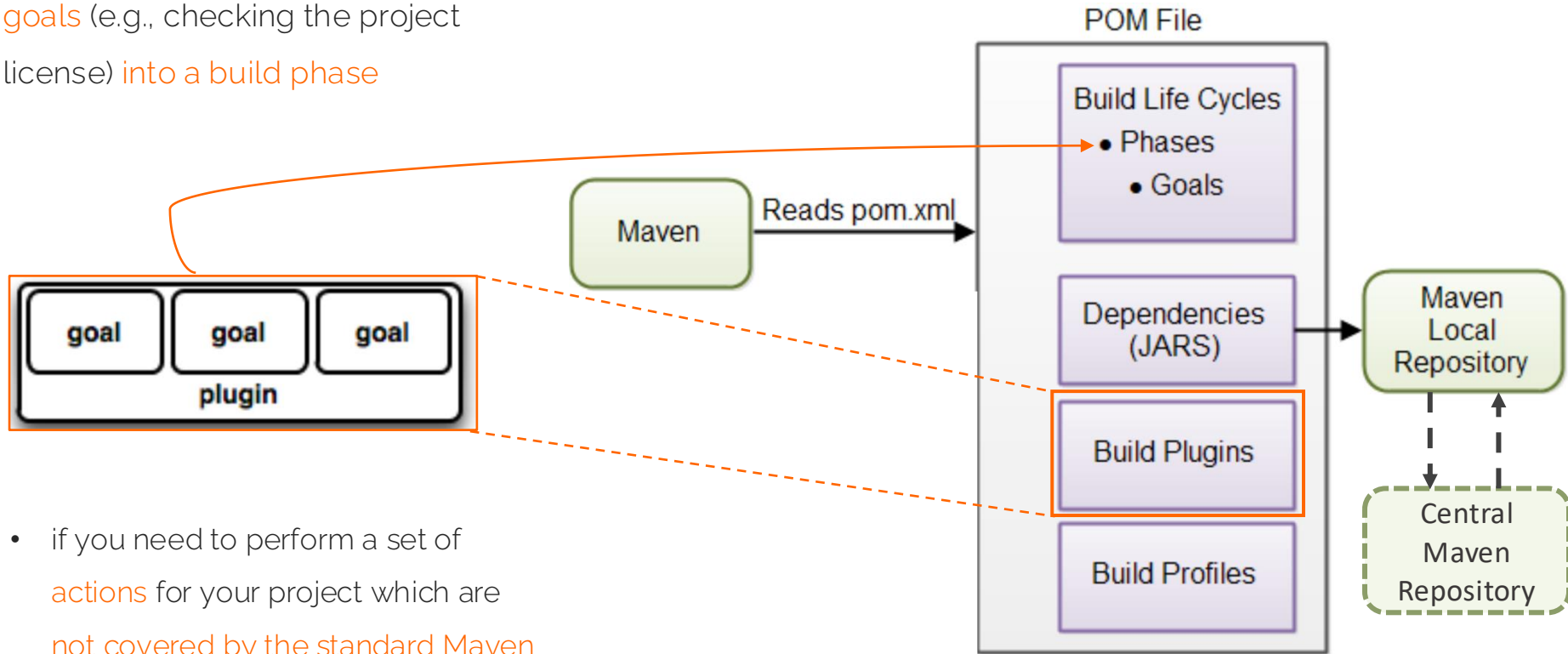
You can also **specify which remote repository** to use for downloading dependencies

More on that later ...



Build Plugins

Build plugins are used to insert **extra goals** (e.g., checking the project license) **into a build phase**



- if you need to perform a set of **actions** for your project which are **not covered by the standard Maven build phases and goals**, you can add a plugin to the POM file

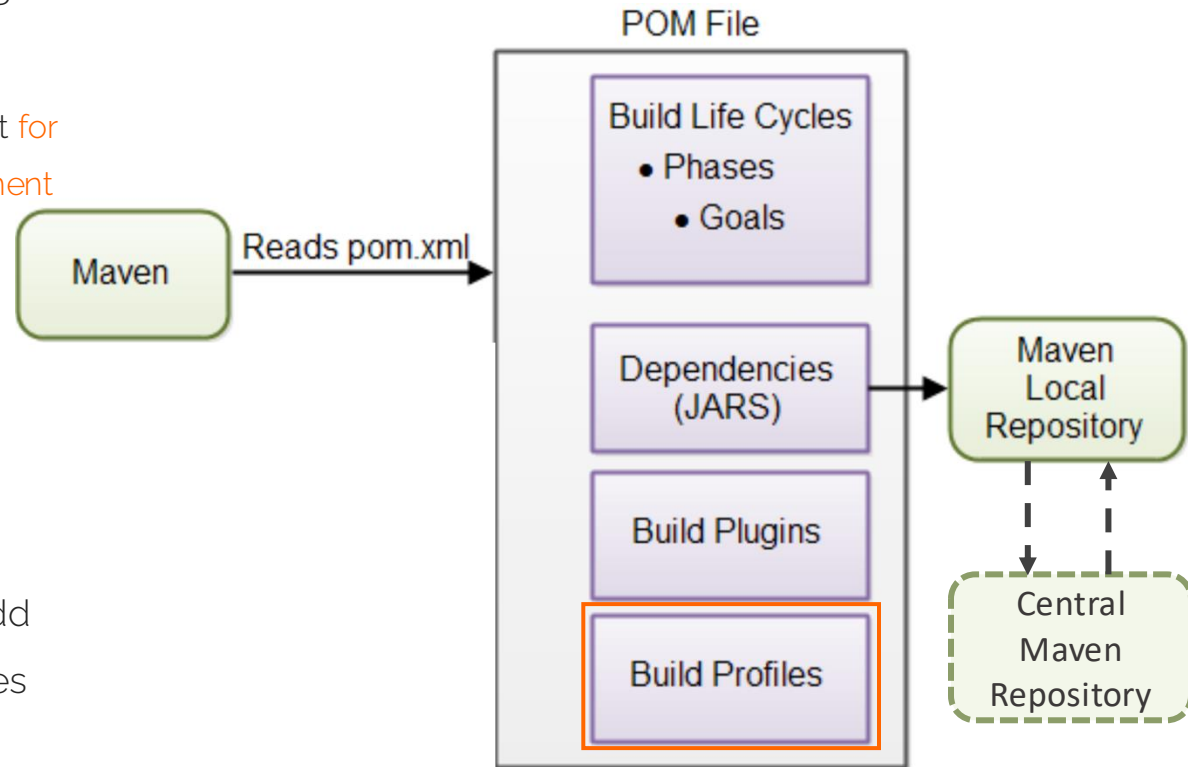
Build Profiles

Build profiles are used if you need to
build your project in different ways (e.g.,
for releasing your project)

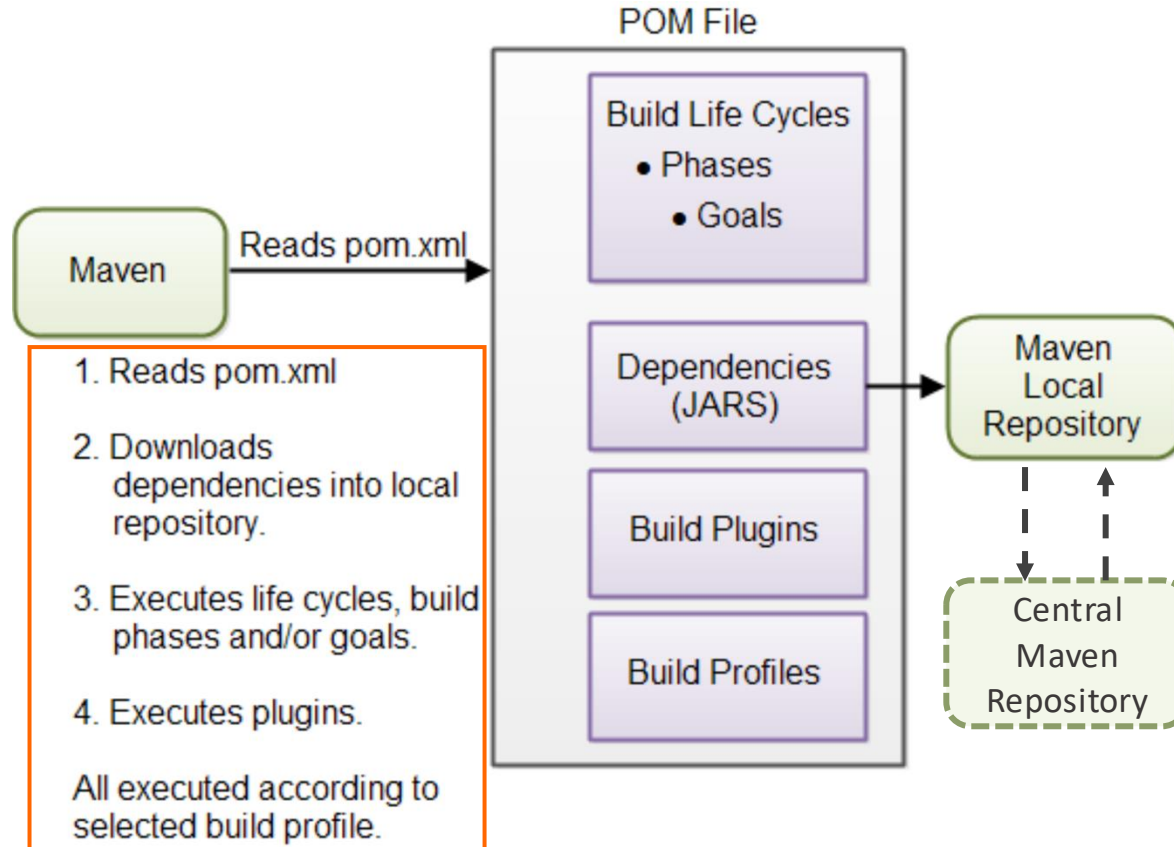
- you may need to build your project for your local computer, for development and test
- you may need to build it for deployment on your production environment

To enable different “builds” you can add different build profiles to your POM files

When executing Maven, you can tell which build profile to use



Maven execution: summing up



Minimal pom

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4                               http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>it.univaq.odws.maven.sum</groupId>
8     <artifactId>SumSOAPServiceMaven</artifactId>
9     <version>1.0.0</version>
10
11     <packaging>war</packaging>
12
13 </project>
```

modelVersion

- set the version of the **POM model** you are using (i.e., the **Super Pom**)
- version 4.0.0 matches Maven version 2.x and 3.x

more on that later...



Minimal pom

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4                               http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>it.univaq.odws.maven.sum</groupId>
8     <artifactId>SumSOAPServiceMaven</artifactId>
9     <version>1.0.0</version>
10
11     <packaging>war</packaging>
12
13 </project>
```

Maven Project's Coordinates

"jar" packing is the default

- a POM requires **groupId**, **artifactId**, and **version** be configured
- these three values form the project's **fully qualified artifact name** in the form of **<groupId>:<artifactId>:<version>**
- it results in a **WAR** file being built and put into the local Maven repository, e.g., at the following path

<userhome>/ .m2/repository/it/univaq/odws/maven/sum/SumSOAPServiceMaven/1.0.0/SumSOAPServiceMaven-1.0.0.war

Build lifecycles, phases and goals

As already anticipated, the **build lifecycle** is divided into **build phases**, in turn divided into **build goals**

Three built-in Build Life Cycles

- **clean**: used for removing temporary files from the output directory
- **default**: used for compiling and packaging your project
- **site**: used for generating documentation for your project

Build Life Cycles

- Phases
- Goals

Each of these build lifecycles **are executed independently** of each other, for instance

- the **clean** phase is part of the **clean lifecycle**
- the **install** phase is part of the **default lifecycle**

You can get Maven to execute more than one build lifecycle, but they will be executed in sequence, separately from each other, as if you had executed two separate Maven commands, e.g., **mvn clean install**

Build lifecycles, phases and goals

Most **commonly** used **build phases** of the **default lifecycle**

the complete list of of the lifecycle phases can be found here:

https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html#Lifecycle_Reference

- **validate**: validates that the project is correct and all necessary information is available (it also makes sure the dependencies are downloaded)
- **compile**: compiles the source code of the project
- **test**: runs the tests against the compiled source code using a suitable unit testing framework
- **package**: packs the compiled code in its distributable format
- **verify**: run any checks to verify the package is valid and meets quality criteria
- **install**: install the package into the local repository, for use as a dependency in other projects locally
- **deploy**: copies the final package to the remote repository for sharing with other developers and projects

Build Life Cycles

- Phases
- Goals

<http://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>

Super Pom

All Maven project POMs extend the **Super POM**, which defines a set of “defaults” shared by all projects

It is part of the Maven installation

This year, I'm using version 3.9.9

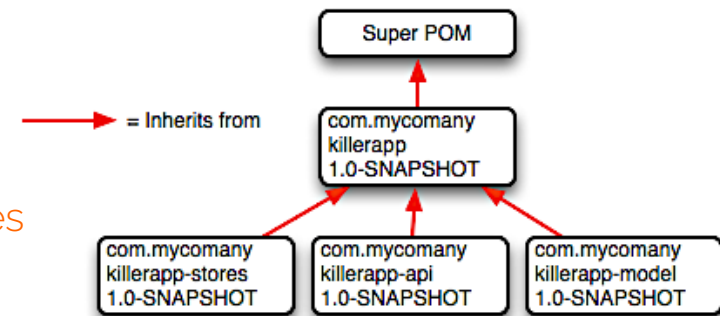
It can be found in **apache-maven-3.6.3/lib/maven-model-builder-3.6.3.jar**

- It may change depending on the Maven version



If you look in this JAR file, you will find a file named **pom-4.0.0.xml** under the **org.apache.maven.model** package... see next slide

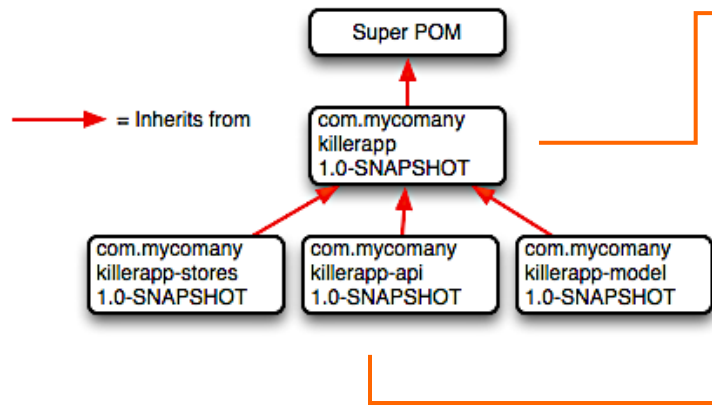
To some extent, it is similar to **java.lang.Object** for Java classes



pom-4.0.0.xml

```
pom-4.0.0.xml > No Selection
22 <!-- START SNIPPET: superpom -->
23 <project>
24   <modelVersion>4.0.0</modelVersion>
25
26   <repositories>
27     <repository>
28       <id>central</id>
29       <name>Central Repository</name>
30       <url>https://repo.maven.apache.org/maven2</url>
31       <layout>default</layout>
32       <snapshots>
33         <enabled>false</enabled>
34       </snapshots>
35     </repository>
36   </repositories>
37
38   <pluginRepositories>
39     <pluginRepository>
40       <id>central</id>
41       <name>Central Repository</name>
42       <url>https://repo.maven.apache.org/maven2</url>
43       <layout>default</layout>
44       <snapshots>
45         <enabled>false</enabled>
46       </snapshots>
47       <releases>
48         <updatePolicy>never</updatePolicy>
49       </releases>
50     </pluginRepository>
51   </pluginRepositories>
52
53   <build>
54     <directory>${project.basedir}/target</directory>
55     <outputDirectory>${project.build.directory}/classes</outputDirectory>
56     <finalName>${project.artifactId}-${project.version}</finalName>
57     <testOutputDirectory>${project.build.directory}/test-classes</testOutputDirectory>
58     <sourceDirectory>${project.basedir}/src/main/java</sourceDirectory>
59     <scriptSourceDirectory>${project.basedir}/src/main/scripts</scriptSourceDirectory>
60     <testSourceDirectory>${project.basedir}/src/test/java</testSourceDirectory>
61     <resources>
62       <resource>
63         <directory>${project.basedir}/src/main/resources</directory>
64       </resource>
65     </resources>
66     <testResources>
67       <testResource>
68         <directory>${project.basedir}/src/test/resources</directory>
69       </testResource>
```

Parent Pom



```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>

    <groupId>com.mycomany</groupId>
    <artifactId>killerapp</artifactId>
    <version>1.0-SNAPSHOT</version>
    <relativePath>../pom.xml</relativePath>

  </parent>

  <artifactId>killerapp-api</artifactId>

  ...

</project>
```

A project divided into subprojects will typically have **one POM file for the parent project**, and **one POM file for each subproject**

- It allows both the total project to be built in one step, or any of the subprojects to be built separately

An inheriting POM file may **override** settings from a **super POM** and a **parent POM**

Dependencies

```
<dependencies>
```

```
  <dependency>
```

```
    <groupId>org.jsoup</groupId>
```

```
    <artifactId>jsoup</artifactId>
```

```
    <version>1.7.1</version>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>junit</groupId>
```

```
    <artifactId>junit</artifactId>
```

```
    <version>4.8.1</version>
```

```
    <scope>test</scope>
```

```
  </dependency>
```

```
</dependencies>
```

- Maven checks the dependencies specified for your project
- Dependencies specify **external libraries** that your project uses
- If the external libraries are **not found in the local Maven repository**, Maven **downloads them from a central Maven repository** and puts them **in your local repository**
- If any of these external libraries need other libraries, they will also be downloaded **recursively**
- You **can also specify which remote repository** to use for downloading dependencies

Snapshot Dependencies

Snapshot dependencies are dependencies (JAR files) which are under development

- a SNAPSHOT version is the **development' version** before the **final release version**:

the SNAPSHOT is always "older" than its release

- Maven periodically checks for updated SNAPSHOT versions (**daily by default**) and downloads them only if they have changed

Globally into **settings.xml**

```
<profiles>
  <profile>
    <repositories>
      <repository>
        <id>central</id>
        <url>https://repo.maven.apache.org/maven2</url>
        <snapshots>
          <updatePolicy>always</updatePolicy>
        </snapshots>
      </repository>
    </repositories>
  </profile>
</profiles>
```

For a specific project into **pom.xml**

```
<repositories>
  <repository>
    <id>my-repo</id>
    <url>https://repo.example.com/maven2</url>

    <snapshots>
      <enabled>true</enabled>
      <updatePolicy>always</updatePolicy>
    </snapshots>

    <releases>
      <updatePolicy>daily</updatePolicy>
    </releases>
  </repository>
</repositories>
```

It can appear under:
<snapshots> for SNAPSHOT dependencies
<releases> for release versions

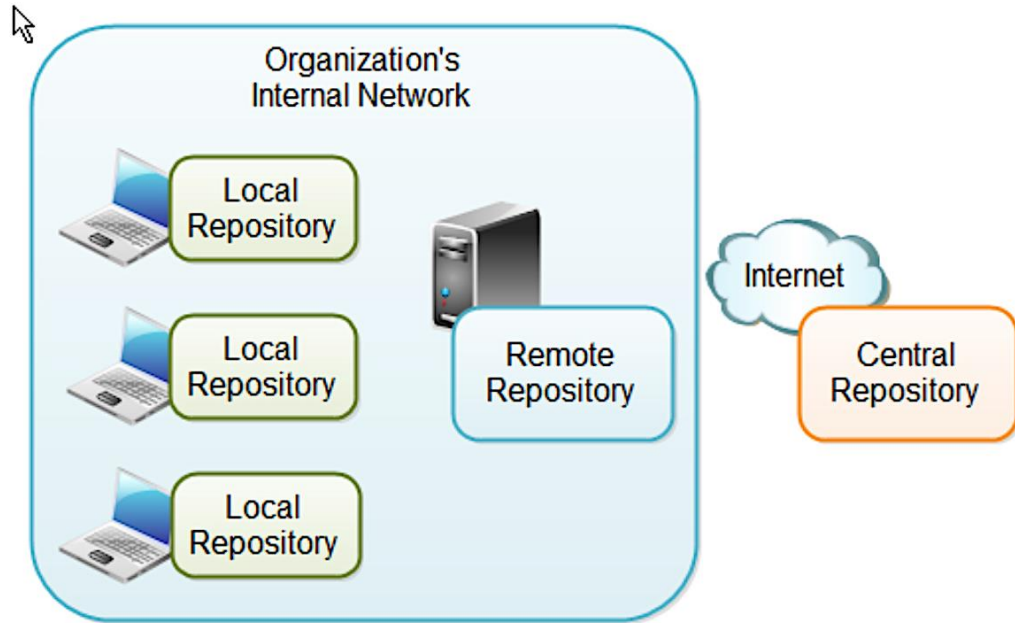
More on Maven repositories

A Maven repository is a **collection of project artifacts** stored in a directory structure that closely matches a project's Maven coordinates

For instance, visit

<https://repo1.maven.org/maven2/>

Maven repositories can be classified as **local** and **remote** (including **Maven Central**). Maven first checks the local repository, then queries remote repositories if needed.



The standard for a repository is to store an artifact in the following directory relative to the root of the repository

`/<groupId>/<artifactId>/<version>/<artifactId>-<version>.<packaging>`

For instance, an artifact having coordinates `org.apache.commons:commons-email:1.1` is available under the directory `/org/apache/commons/commons-email/1.1/` in a file named `commons-email-1.1.jar`

<https://repo1.maven.org/maven2/>

<https://mvnrepository.com/repos/central>

Local Repository

It is a directory on the developer's computer. This repository will contain all the dependencies Maven downloads

Your own projects can also be built and installed in your local repository, using the `mvn install`. That way your other projects can use the packaged JAR files of your own projects as external dependencies by specifying them as external dependencies inside their Maven POM files

By default, Maven puts your local repository inside your user home directory on your local computer, i.e., `<user-home>/ .m2/repository` (on Unix-like systems: `~/.m2/repository`)

You can change the location of the local repository by setting the directory inside your Maven settings file `<user-home>/ .m2/settings.xml` (on Unix-like systems: `~/.m2/settings.xml`)

```
<settings>
  <localRepository>
    d:\data\java\products\maven\repository
  </localRepository>
</settings>
```

Central and Remote Repository

Central Repository (it is a remote repository by default)

- it is a repository **provided by the Maven community**: by default, Maven looks in this central repository for any dependencies needed but not found in your local repository

Remote Repository

- it is a **repository on a web server from which Maven can download dependencies**
- A remote repository can be located anywhere on the internet, or inside a local network
- a remote repository is often used **for hosting projects internal to an organization**, which are shared by multiple projects
- put the following XML elements in the pom.xml right after the <dependencies> element (it can also be don globally into **settings.xml**)

```
<repositories>
  <repository>
    <id>disim.code</id>
    <url>http://maven.disim.univaq.com/maven2/lib</url>
  </repository>
</repositories>
```

Download and install Maven

Download the `binary` zip archive from <https://maven.apache.org/download.cgi>

Install (see next slide)

- ensure `JAVA_HOME` environment variable is set
- extract the archive to a suitable location
- add `apache-maven-3.9.9/bin` directory to the `PATH` environment variable

Check if maven works

- open a command prompt and type `mvn -version`

Maven on Linux or MacOS

To install Maven on a POSIX Compliant Operating System, such as Linux or MacOS, perform the following steps:

- Ensure JAVA_HOME is set: `echo $JAVA_HOME`
- Download the binary archive from a download mirror and expand it, e.g.,
`/opt/apache-maven-3.9.9` or `/usr/local/apache-maven-3.9.9`
- Add Maven binaries to the `PATH` (see next slide)
- Open a new terminal window and type this command

`mvn -v` or `mvn -version`

Setting PATH on Linux or MacOS

- Open up Terminal and run the following command

```
sudo nano /etc/paths
```

- Enter your password, when prompted
- Go to the bottom of the file, and enter the path you wish to add
- Hit control-x to quit
- Enter "Y" to save the modified buffer
- That's it! To test it, in new terminal window, type

```
marcoautili@iMac-di-User ~ % cat /etc/paths
/usr/local/bin
/System/Cryptexes/App/usr/bin
/usr/bin
/bin
/usr/sbin
/sbin
/opt/apache-maven-3.9.9/bin ←
marcoautili@iMac-di-User ~ %
```

```
echo $PATH
```

- Similarly to what I did to set the JAVA_HOME variable permanently, you can also opt for adding the PATH configuration to a script that will run every time you login
- To do this, add the following lines to, e.g., .bash login, .profile or .zprofile

```
export PATH=/usr/local/maven/bin:$PATH
```

Maven on Window

Installing Maven on Windows is very similar to installing Maven on Mac OS X, the main differences being the installation location and the setting of an environment variable

- **unpack** Maven to the installation directory, e.g., `C:\Program Files\apache-maven-3.9.9`
- **update the PATH** environment variable similarly to what already done for JAVA

Be sure your JAVA_HOME is properly set

As already said, the JAVA_HOME environment variable must be properly set so that Maven knows **where to find the JDK commands to execute**

- Maven goals run the appropriate Java commands against the project's source code
 - For example, the most common scenario is to compile the code by executing the **javac** command (remember: **javac** is part of the **JDK** and **not the JRE**)
- If **JAVA_HOME** is not pointing to a valid **JDK** installation, Maven will throw an error at every execution. Check for **JAVA_HOME** by typing:
 - **mvn -version**

```
marcoautili@Marcos-MacBook-Pro ~ %  
marcoautili@Marcos-MacBook-Pro ~ % echo $JAVA_HOME
```

```
marcoautili@Marcos-MacBook-Pro ~ % mvn -v  
The operation couldn't be completed. Unable to locate a Java Runtime.  
Please visit http://www.java.com for information on installing Java.
```

```
marcoautili@Marcos-MacBook-Pro ~ %  
[marcoautili@Marcos-MacBook-Pro ~ %  
[marcoautili@Marcos-MacBook-Pro ~ % which mvn  
/opt/apache-maven-3.9.9/bin/mvn  
[marcoautili@Marcos-MacBook-Pro ~ %  
[marcoautili@Marcos-MacBook-Pro ~ % mvn -v  
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)  
Maven home: /opt/apache-maven-3.9.9  
Java version: 21.0.6, vendor: Oracle Corporation, runtime: /Library/Java  
/JavaVirtualMachines/jdk-21.jdk/Contents/Home  
Default locale: en_IT, platform encoding: UTF-8  
OS name: "mac os x", version: "15.2", arch: "aarch64", family: "mac"  
[marcoautili@Marcos-MacBook-Pro ~ %
```

Setting JAVA_HOME on my Mac (from first lecture)

- I created `~/.zprofile` (i.e., `/Users/marcoautili/.zprofile`) since user accounts in MacOS Catalina and later Mac OS default to using `zsh`, and not `bash` anymore

- After updating to Catalina, I do not have the file `~/.bash_profile` anymore

- Open up Terminal
- Run the following command



`sudo nano ~/.zprofile`

Create `~/.zprofile` if it does not exist already (e.g., `touch ~/.zprofile`)

- Enter your password, when prompted
- Go to the bottom of the file, and enter the path you wish to add
- Hit `control-x` to quit
- Enter `"Y"` to save the modified buffer

`export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-21jdk/Contents/Home`

IMPORTANT: for the file `.zprofile` to be read each time you open the terminal, make sure that, for the current user account, `zsh` is the default shell. To update your account to **use `zsh` by default**:

```
chsh -s /bin/zsh
```

```
marcoautili@iMac-di-User ~ %  
marcoautili@iMac-di-User ~ % cat ~/.zprofile  
export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home  
marcoautili@iMac-di-User ~ %
```



Maven in my machine

